50% implementation report

Project Title

Graph Based Text to Floorplan conversion with 3d visualization

Abstract

This project is for conversion of natural language to fully 3d visualized floorplans for architecting and designing living spaces without special design skills or tools.I use a graph based method for easy semantic conversion from text to floorplan.

Background

Existing projects which aim to this use rule based approach or image GANs which don’t understand geometry or special attributes which the user wants.What this project will do is a graph based approach with geometry predictions according to the prompt of the user. The graphs have Nodes as rooms and doors as edges so every floorplan can be represented as a graph with edge features denoting direction of the connection.

Proposed System

First the user prompt for example

“Need a 2bhk apartment with spacious living “

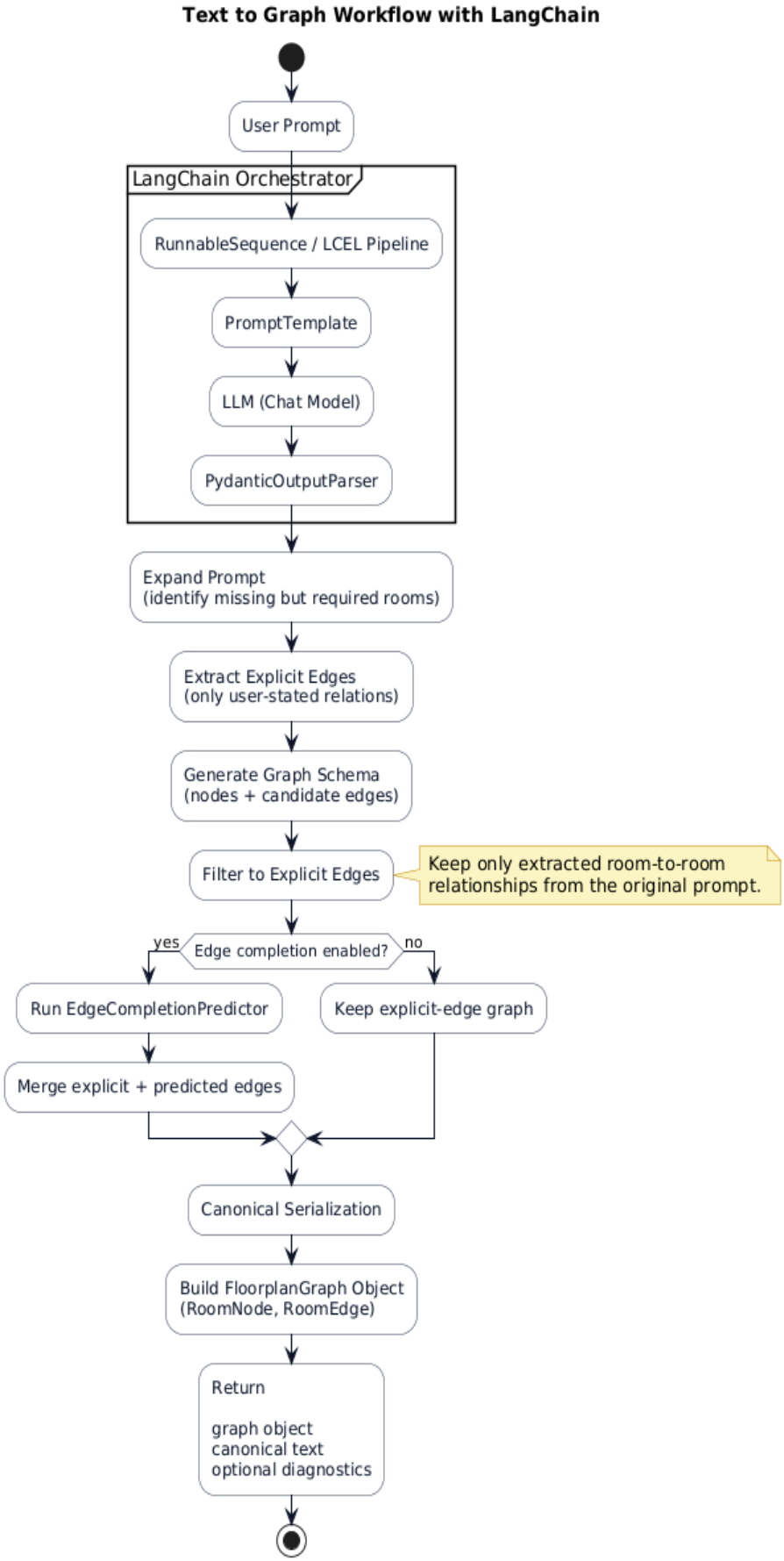
We use orchestration of agents with the help of Langchain to convert the prompt to a fully viable floorplan.

The first agent expands the prompt to include more rooms so that a valid floorplan can be created.

“

Original rooms: ['living', 'kitchen', 'bedroom', 'bathroom', 'front_door']
Added rooms: []
All rooms: ['living', 'kitchen', 'bedroom', 'bathroom', 'front_door']
Reasoning: The user's prompt already includes all essential rooms for a complete residential floorplan: at least one living room, one kitchen, two bedrooms (fulfilling the minimum of one bedroom), one bathroom, and one front door. No additional rooms are necessary for a functional and complete 2BHK apartment.
Spatial context: The layout suggests a compact, efficient design where the living room acts as the central hub, with the kitchen adjacent for easy access. The two bedrooms are likely private spaces away from common areas, and the single bathroom connects to the central living or hallway. The front door opens into the living area, creating a natural flow into the home

“


Text to Graph Workflow with LangChain
User Prompt
LangChain Orchestrator
RunnableSequence / LCEL Pipeline
PromptTemplate
LLM (Chat Model)
PydanticOutputParser
Expand Prompt
(identify missing but required rooms)
Extract Explicit Edges
(only user-stated relations)
Generate Graph Schema
(nodes + candidate edges)
Filter to Explicit Edges
Keep only extracted room-to-room relationships from the original prompt.
Edge completion enabled?
yes
no
Run EdgeCompletionPredictor
Keep explicit-edge graph
Merge explicit + predicted edges
Canonical Serialization
Build FloorplanGraph Object
(RoomNode, RoomEdge)
Return
graph object
canonical text
optional diagnostics

The next agent extracts the explicitly mentioned edges or connections from the prompt.

Example:

“Design a modern house with living room directly connected to a gaming room”

Here the edge living room -> gaming room

Then a basic graph is constructed with rooms as nodes and explicitly mentioned edges.

Extracted explicit edges:

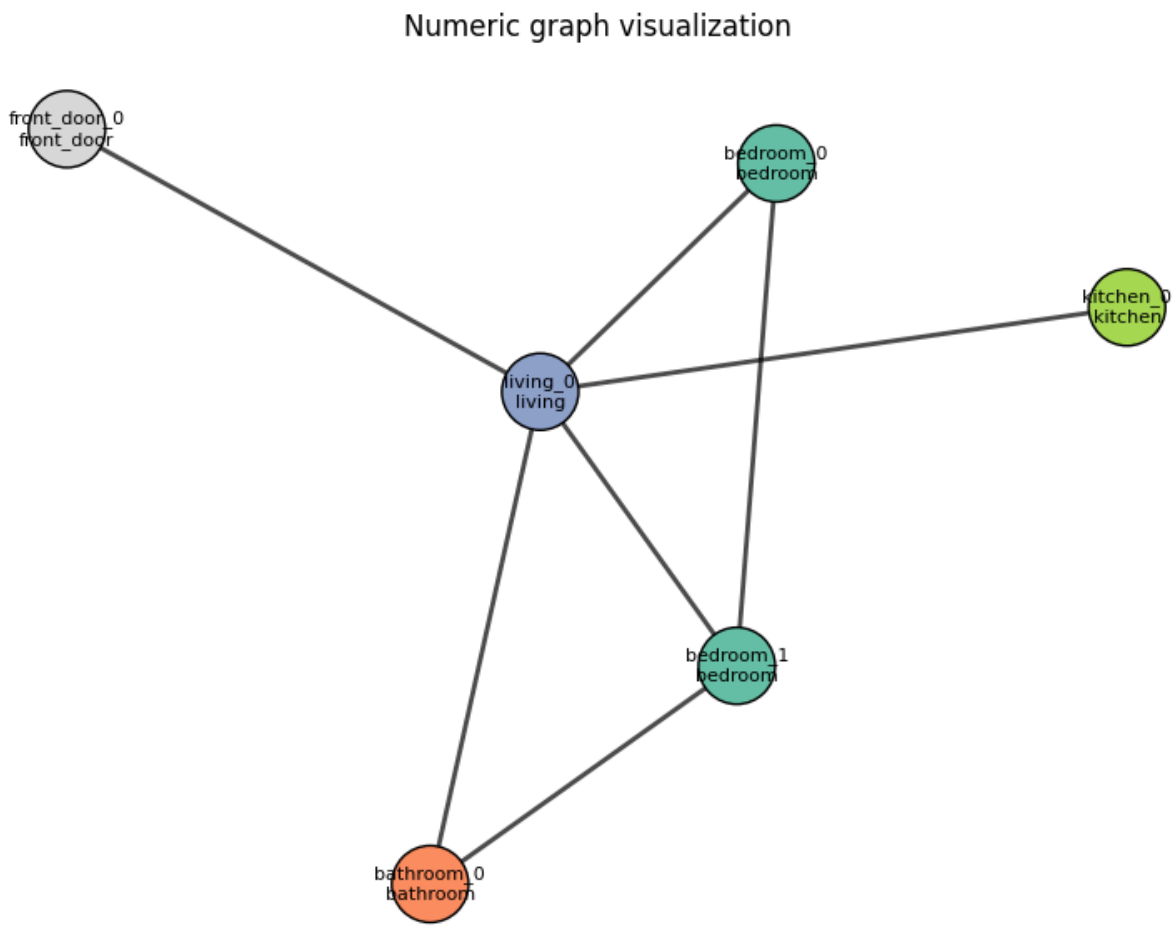
- kitchen -> living (relation: adjacency, direction: None)

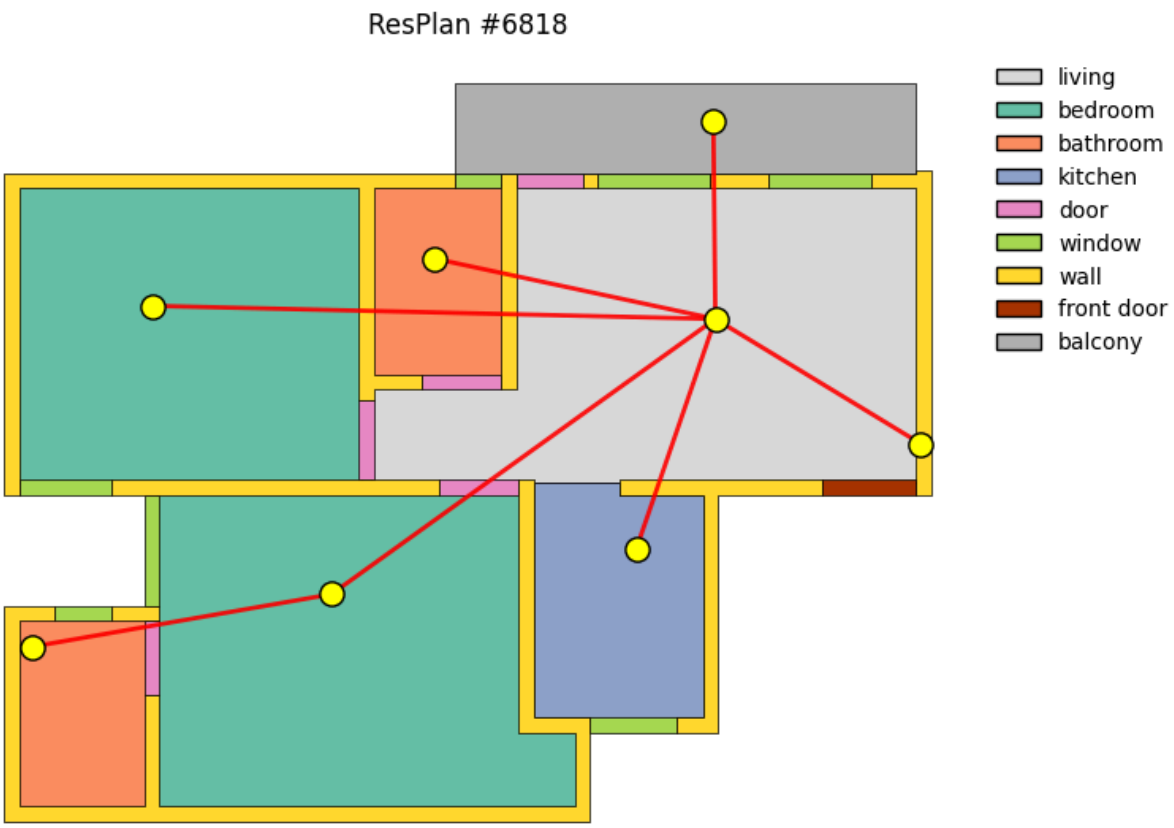
- front_door -> living (relation: connected_to, direction: None)

Rooms in graph schema (before completion):
- bedroom_0: bedroom
- bedroom_1: bedroom
- bathroom_0: bathroom
- front_door_0: front_door
- kitchen_0: kitchen
- living_0: living

Edges in graph schema (before completion):
- living_0 --[adjacency]--> kitchen_0
- front_door_0 --[connected_to]--> living_0

Then we use a GNN Graph neural network with some rule based connections to connect all nodes. It makes sure no room is completely isolated and rooms are connected logically like bedrooms are connected to bathrooms.


Numeric graph visualization
front_door_0
front_door
bedroom_0
bedroom
kitchen_0
kitchen
living_0
living
bedroom_1
bedroom
bathroom_0
bathroom



This is how graph and floorplans are related.

Training

We train using the resplan dataset which has 17000 floorplans with metadata about rooms polygons.

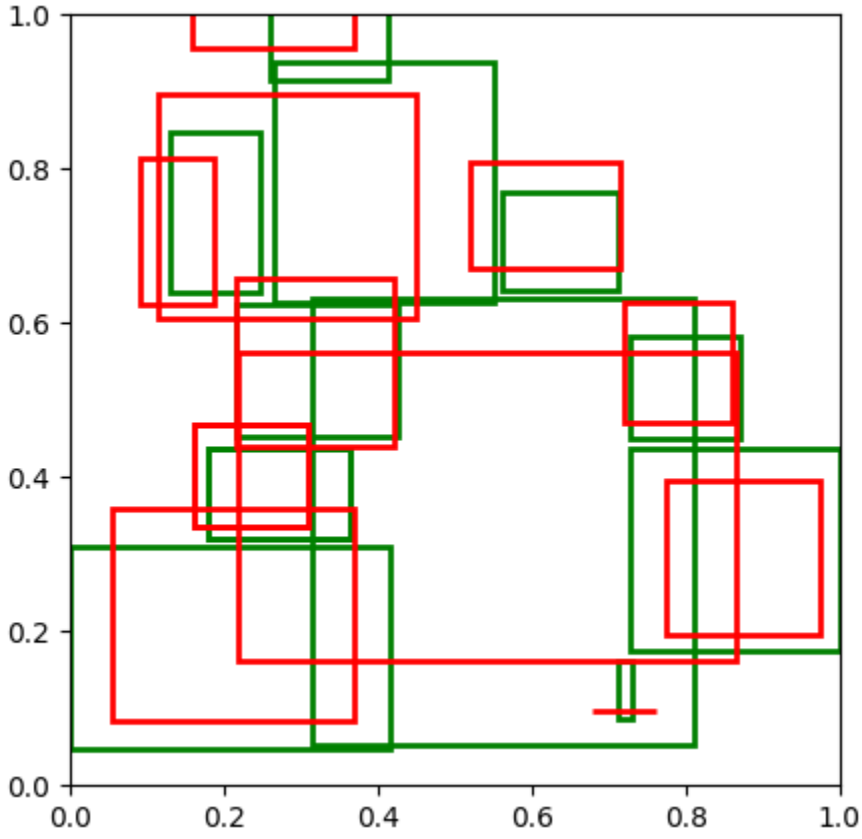
We train this model three GINEConv layers

```
        self.conv1 = GINEConv(mlp(node_dim, hidden), edge_dim=edge_dim)
        self.conv2 = GINEConv(mlp(hidden, hidden), edge_dim=edge_dim)
        self.conv3 = GINEConv(mlp(hidden, hidden), edge_dim=edge_dim)
```

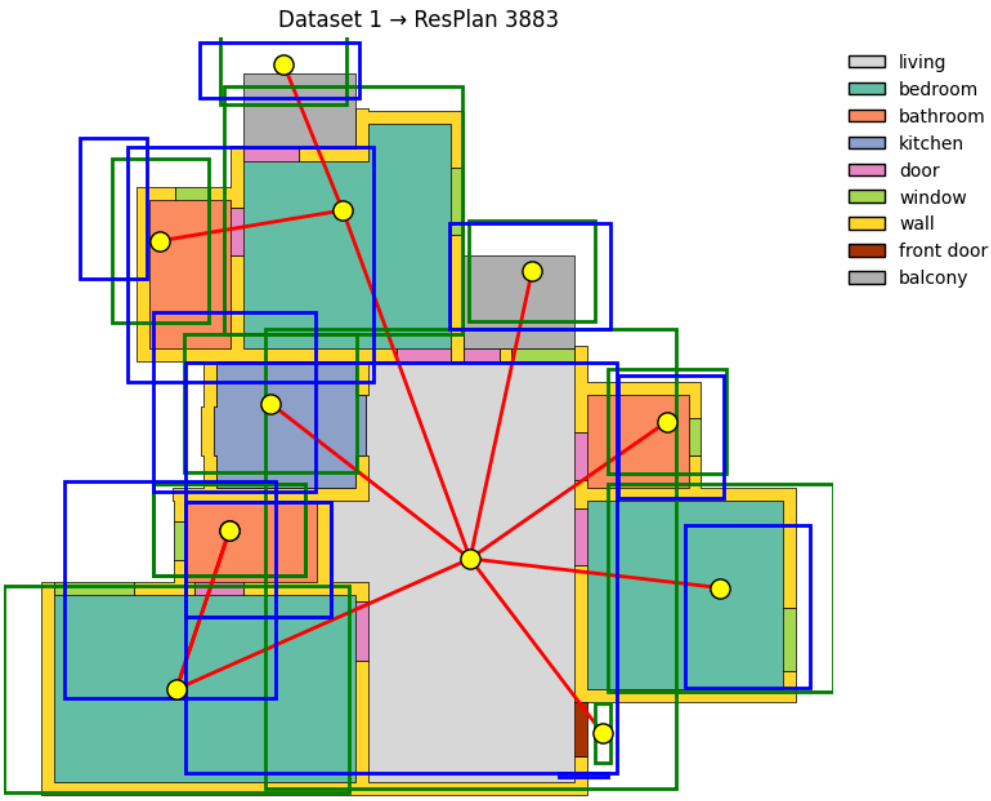
This captures the type of room , its centroid how it is related to other rooms and its x and y coordinates .

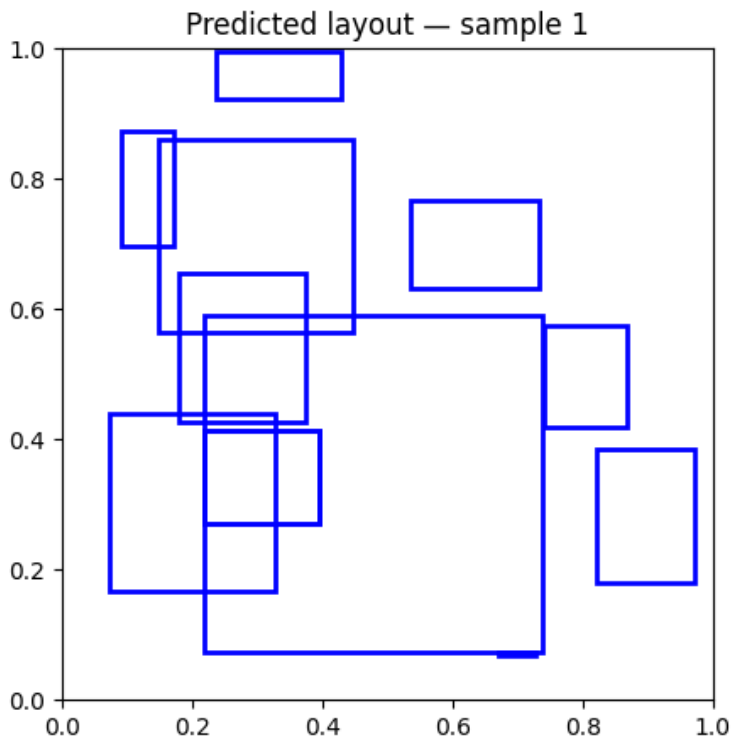
This model outputs the x,y area and centroid of each room.

Plotting this comes like this



Here the green represents the ground truth and the red represents the predicted rooms.

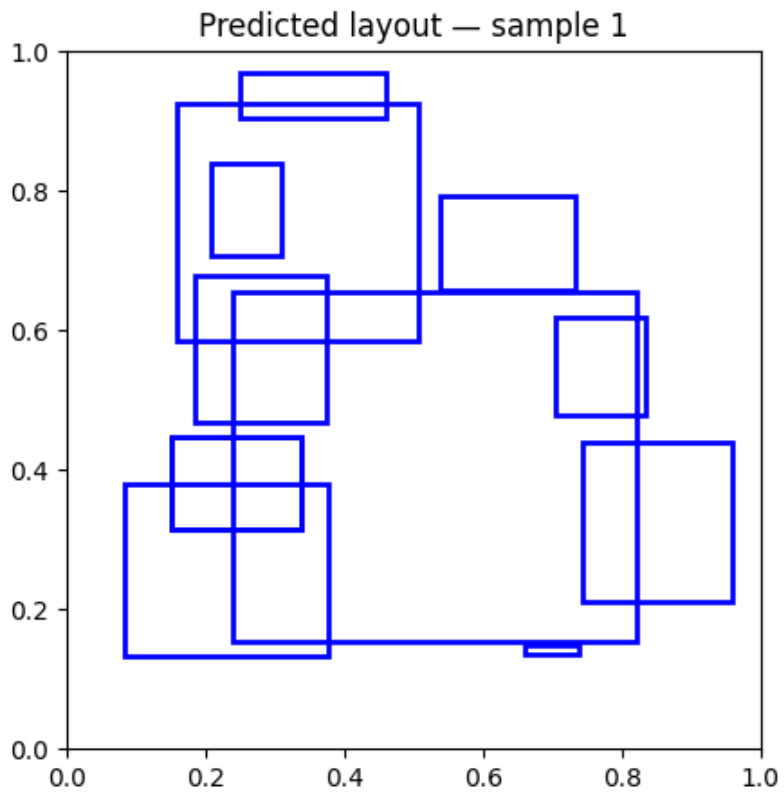

Dataset 1 → ResPlan 3883
living
bedroom
bathroom
kitchen
door
window
wall
front door
balcony


Predicted layout — sample 1
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

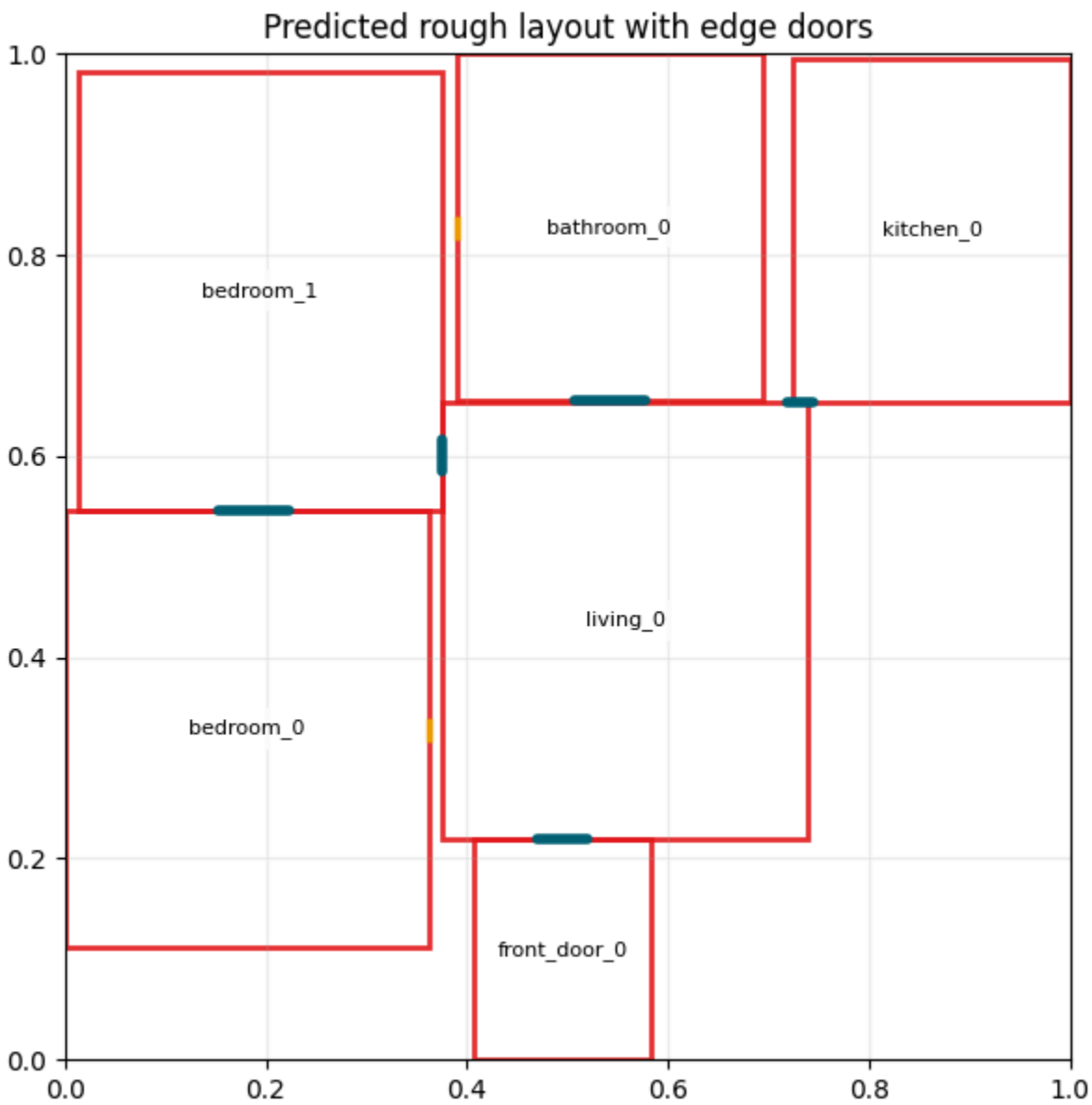
To improve connectivity and layout validity we add 3 losses when training

- Layout loss
- Adjacency loss
- Overlap loss

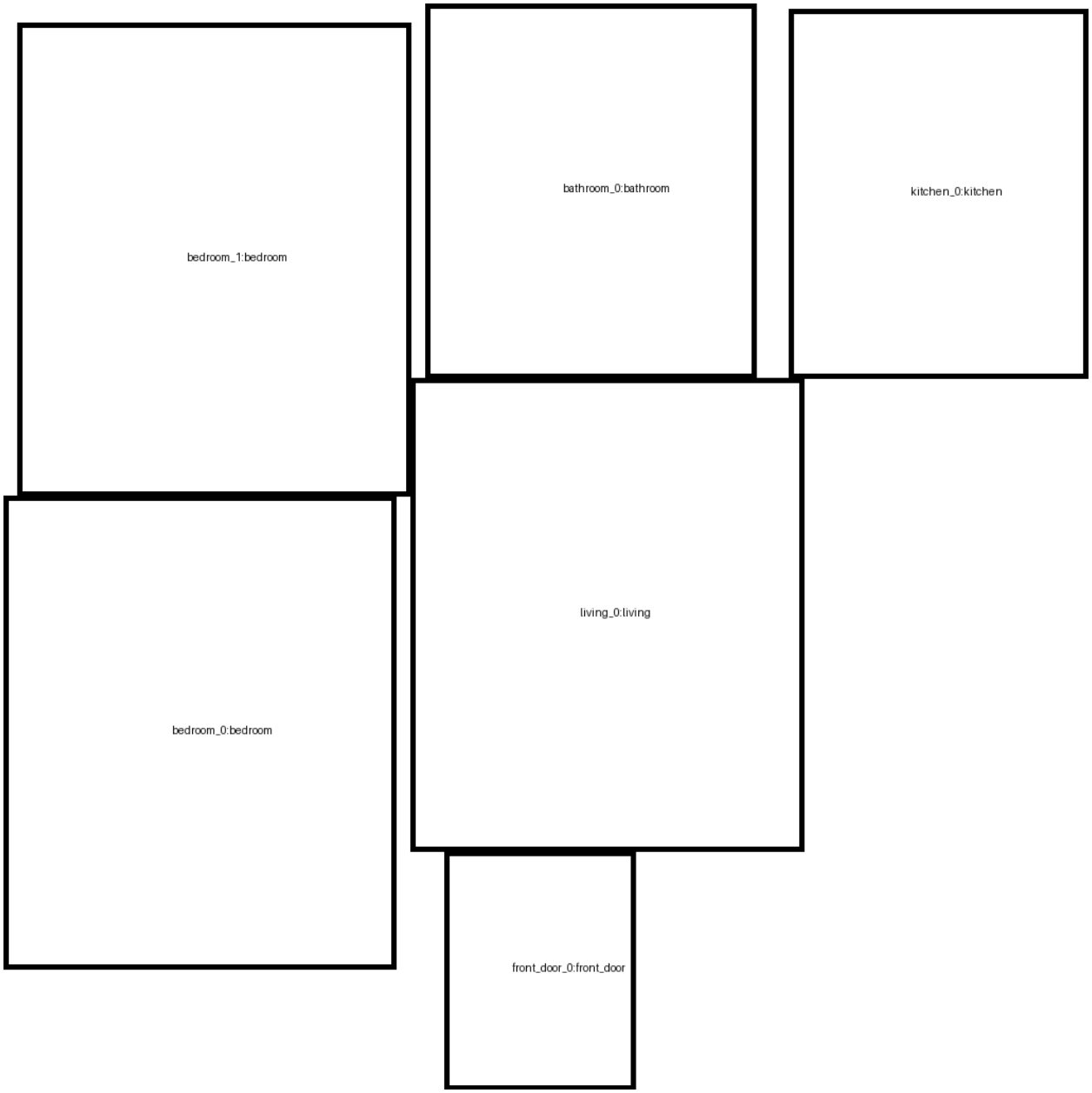
This increases the validity of the predicted layout



This says the approximate prediction is then glued together and fixed the final layer to create the final 2d layout



Then stable diffusion using control net is used to form valid floorplan with a polished look



bedroom_1:bedroom
bathroom_0:bathroom
kitchen_0:kitchen
living_0:living
bedroom_0:bedroom
front_door_0:front_door